

Forge — Executive Summary

An AI-native developer platform for Cardano. Describe a smart contract in plain English; get back a real, compiled Aiken project, a typed TypeScript SDK, passing tests, and a deployment artifact — in about ten seconds, verified against the real Aiken compiler.

The problem

Cardano has excellent individual tools — Aiken, Lucid Evolution, Mesh, Blockfrost — but no project owns the integration between them. Every team hand-assembles project structure, hand-writes TypeScript types with no compiler-enforced link back to the validator, hand-rolls deployment tracking, and has no systematic check for eUTxO-specific bugs like double satisfaction. None of this is a flaw in the individual tools — it's a gap between them.

The solution

`forge build "<description>"` — one command — takes a natural-language request and runs it through a real pipeline: classify intent → confirm confidence → render Aiken source from an audited template → compile with the real `aiken` binary → parse the CIP-57 blueprint → generate a typed TypeScript SDK → run a functional test → compute a real CIP-19 deployment address. Three templates exist today: **Escrow with Milestone Payments**, **NFT Minting with Royalties**, and **Token Vesting** — each hand-audited Aiken source, not AI-generated.

Why it's trustworthy, not just impressive

The language model never writes blockchain logic. It has exactly two responsibilities anywhere in the platform: classify intent, and narrate decisions the system already made deterministically. A separate, deterministic template engine renders every line of Aiken source. A description that doesn't confidently match any of the three templates is **rejected outright** — no project is scaffolded, no low-confidence guess is ever generated. See [ADR-003](#) and [ADR-006](#).

What's real, verified today

- Real Aiken compilation and CIP-57 blueprint parsing (not mocked)
- 3 audited, compiling contract templates covering distinct validator shapes (`spend` and `mint`)
- Real typed TypeScript SDK generation from the compiled blueprint

- Real CIP-19 bech32 deployment address computation
- Confidence-gated template selection with a clear, actionable rejection error
- 150 fast unit tests + 8 real integration tests (real Aiken compiler, real network), all passing, verified from a fully clean workspace

Honest gaps (disclosed, not discovered)

The single largest gap versus the original pitch: `ai-testgen`, a deterministic eUTxO vulnerability rule engine, is architected and wired into the pipeline but has no rule engine registered yet — `forge build` correctly reports zero findings rather than fabricating any. Real off-chain transaction building, a local devnet, and additional presentation layers (VS Code, web) are roadmap, not built. Full, unhedged assessment in [docs/ProductionReadiness.md](#).

Try it yourself in under a minute

```
pnpm install && pnpm build
node packages/cli/dist/bin.js build "Build an escrow smart contract with milestone-based payments"
```

Learn more

[README.md](#) · [docs/JudgeCheatSheet.md](#) · [docs/JudgePreparation.md](#) · [docs/Architecture.md](#)